HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and Communications Technology

Software Design Document

Version 1.3

AIMS – An Internet Media Store

Subject: ITSS Software Development

Group 23

Bùi Trung Hiếu – 20235932
Nguyễn Trần Bình – 20235899
Trần Trung Nghĩa – 20235983 (PIC: View Product Detail)
Nguyễn Hữu Tuấn Tú – 20236005
Nguyễn Tuấn Khanh – 20235952

Hanoi, June 2026

Table of Contents

1 Introduction

This Software Design Document (SDD) describes the software design of the AIMS – An Internet Media Store system. It translates the requirements stated in the Software Requirements Specification (SRS v1.3) into an architectural and detailed design that can be directly implemented and verified. The present version consolidates the design of the system for Lab 13 and provides an in-depth elaboration of the View Product Detail feature (use case UC235), which is the responsibility of the person in charge (PIC) Trần Trung Nghĩa.

1.1 Objective

The purpose of this document is to provide a complete and unambiguous design reference for the AIMS system so that developers can implement the software, testers can derive test cases, and reviewers can assess design quality. It defines the system architecture, the analysis and design class models, the user interface design, the data model, and the design-quality considerations (coupling, cohesion, SOLID principles, and design patterns). The intended audience comprises the development team, the course instructors, and reviewers responsible for evaluating the AIMS design. While the document covers the system as a whole, it elaborates the View Product Detail feature in full depth as the assigned scope of this submission.

1.2 Scope

The software product specified by this document is AIMS – An Internet Media Store, an e-commerce web application that enables customers to browse, search, and purchase physical media products (Book, Newspaper, CD, and DVD) and enables Product Managers and Administrators to manage products, orders, and user accounts. The system is implemented as a monorepo composed of a NestJS (TypeScript) back-end exposing a REST API, an Angular front-end client, and a PostgreSQL database accessed through the Prisma ORM. External services VietQR and PayPal Sandbox support payment.

Within this scope, the View Product Detail feature allows a Customer or a Product Manager to retrieve and display the full information of a selected product, including its general attributes and the attributes specific to its product type. This document gives an executive overview of the whole system and a detailed design of the View Product Detail feature; the detailed design of other features (Place Order, Pay Order, CUD Product) is maintained by their respective PICs and is referenced here only where needed for consistency.

1.3 Glossary

Term	Description
AIMS	An Internet Media Store – the e-commerce system designed in this document.
UC235	The identifier of the View Product Detail use case.
PIC	Person In Charge – the team member responsible for a given feature.
DTO	Data Transfer Object – a read-only object that carries data across layers.
Use Case	A unit of business functionality coordinated by the application layer.
Repository	An abstraction that encapsulates data access for the domain layer.

Term	Description
ORM	Object–Relational Mapping; AIMS uses Prisma over PostgreSQL.
RBAC	Role-Based Access Control – authorization based on user roles.
SPA	Single-Page Application – the Angular client architecture.

1.4 References

- AIMS – Software Requirements Specification (SRS), Version 1.3, Group 23.
- AIMS – Use Case Specification: View Product Detail (UC235), Group 23.
- AIMS source code repository (NestJS API and Angular client), branch release/lab13.
- R. C. Martin, Clean Architecture: A Craftsman’s Guide to Software Structure and Design, 2017.
- E. Gamma et al., Design Patterns: Elements of Reusable Object-Oriented Software, 1994.

2 Overall Description

2.1 General Overview

AIMS follows a layered, clean-architecture organization on the server side and a feature-oriented component architecture on the client side. The server separates responsibilities into an interface layer (NestJS controllers), an application layer (use cases, mappers, and DTOs), a domain layer (entities and repository interfaces), and an infrastructure layer (Prisma repositories and external service adapters). The client is an Angular single-page application composed of feature modules whose components communicate with the server exclusively through dedicated API services.

For the View Product Detail feature, the Customer or Product Manager selects a product from the product list, search result, or home page. The client navigates to the product-detail route, the ProductApiService issues an HTTP GET request to the back-end, the GetProductDetailUseCase validates the identifier and retrieves the product through the ProductQueryRepository abstraction, the ProductDetailMapper transforms the persistence record into a read-only ProductDetailDto, and the client renders the corresponding product-detail screen. This design isolates business orchestration from persistence and presentation, keeping each layer independently testable and replaceable.

2.2 Assumptions/Constraints/Risks

2.2.1 Assumptions

- Each product belongs to exactly one of the supported types: Book, Newspaper, CD, or DVD.
- A product selected for viewing already exists in the catalog at the moment of selection; concurrent deletion is handled by the not-found path.
- Product images are hosted on an external media service (Cloudinary) and referenced by URL.
- The Customer may view active products without authentication; the Product Manager is authenticated before accessing management screens.

2.2.2 Constraints

- Implementation technologies are fixed: NestJS + TypeScript (API), Angular (client), PostgreSQL + Prisma (persistence).
- The maximum response time of the system is 2 seconds under normal load and 5 seconds at peak, as stated in the SRS.
- Product detail data is read-only for this use case; no business data is modified during View Product Detail.
- The REST endpoint for product detail must remain backward compatible: GET /products/:id.

2.2.3 Risks

- Invalid or non-numeric product identifiers supplied through the URL could reach the persistence layer; mitigated by identifier validation in the use case before any query is issued.
- Type-specific mapping logic could grow into a large conditional block as new product types are added; mitigated by the Strategy-based mapping design proposed in Section 5.5.
- Database or media-service unavailability could break the detail screen; mitigated by explicit not-found and error handling and a safe redirect on the client.

3 System Architecture and Architecture Design

The architecture is derived in three steps: (1) selecting architectural patterns that satisfy the quality attributes in the SRS; (2) allocating the responsibilities of each use case to analysis classes of the boundary–control–entity stereotypes; and (3) refining those classes into the layered design classes that are implemented in the source code. This section presents the patterns, the interaction and analysis models for View Product Detail, and the security architecture.

3.1 Architectural Patterns

AIMS adopts a Layered (Clean) Architecture on the server combined with the Model–View–Controller and Repository patterns. The layered organization was chosen because it enforces a strict dependency direction toward abstractions, which keeps the application and domain layers independent of frameworks and databases and therefore highly testable. The Repository pattern decouples the application layer from Prisma, and the DTO/Mapper pattern keeps persistence models from leaking into controllers and the client. On the client, Angular implements a component-based MVC variant where smart page components coordinate state and presentational components render type-specific information.

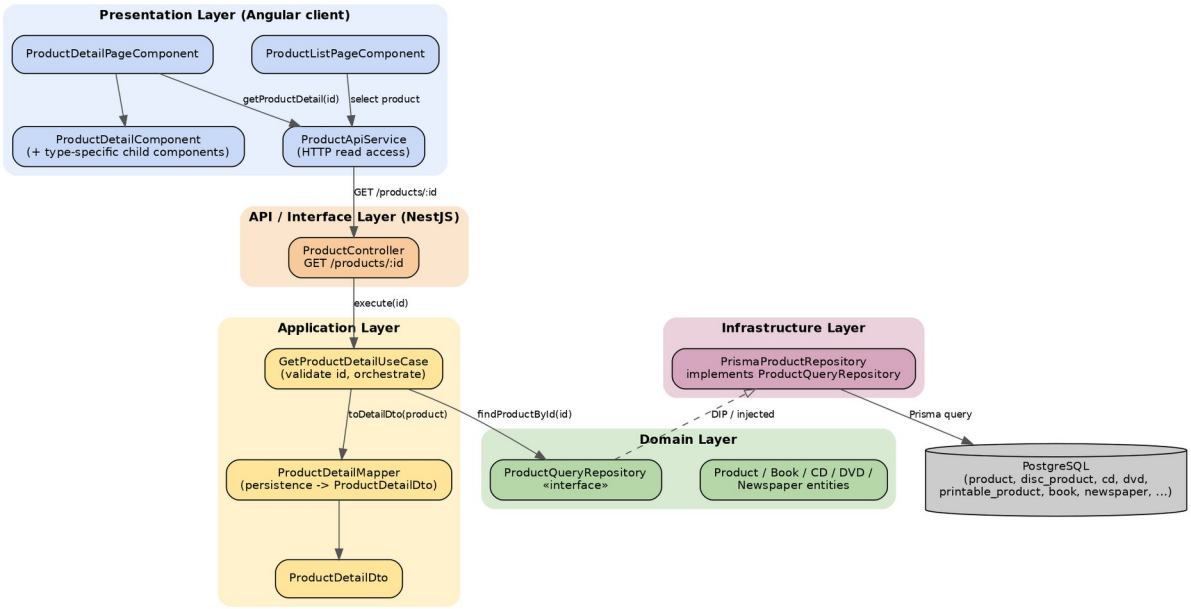

Figure 1. Layered (clean) architecture of the View Product Detail feature

The dependency rule is observed throughout: the interface and application layers depend on the ProductQueryRepository abstraction, while the concrete PrismaProductRepository in the infrastructure layer depends inward on that abstraction. This is the application of the Dependency Inversion Principle at the architectural level.

3.2 Interaction Diagrams

The sequence diagram below models the basic and alternative flows of UC235. The actor selects a product on the ProductListScreen, which invokes the ViewProductDetailController. The controller asks the ProductRepository to find the product, which queries the database.

When the product is not found, the controller produces a “Product not found” response and the client redirects safely. When the product is found and the actor is a Product Manager, the controller verifies authorization before displaying the detail; for a Customer, the active product detail is displayed directly.

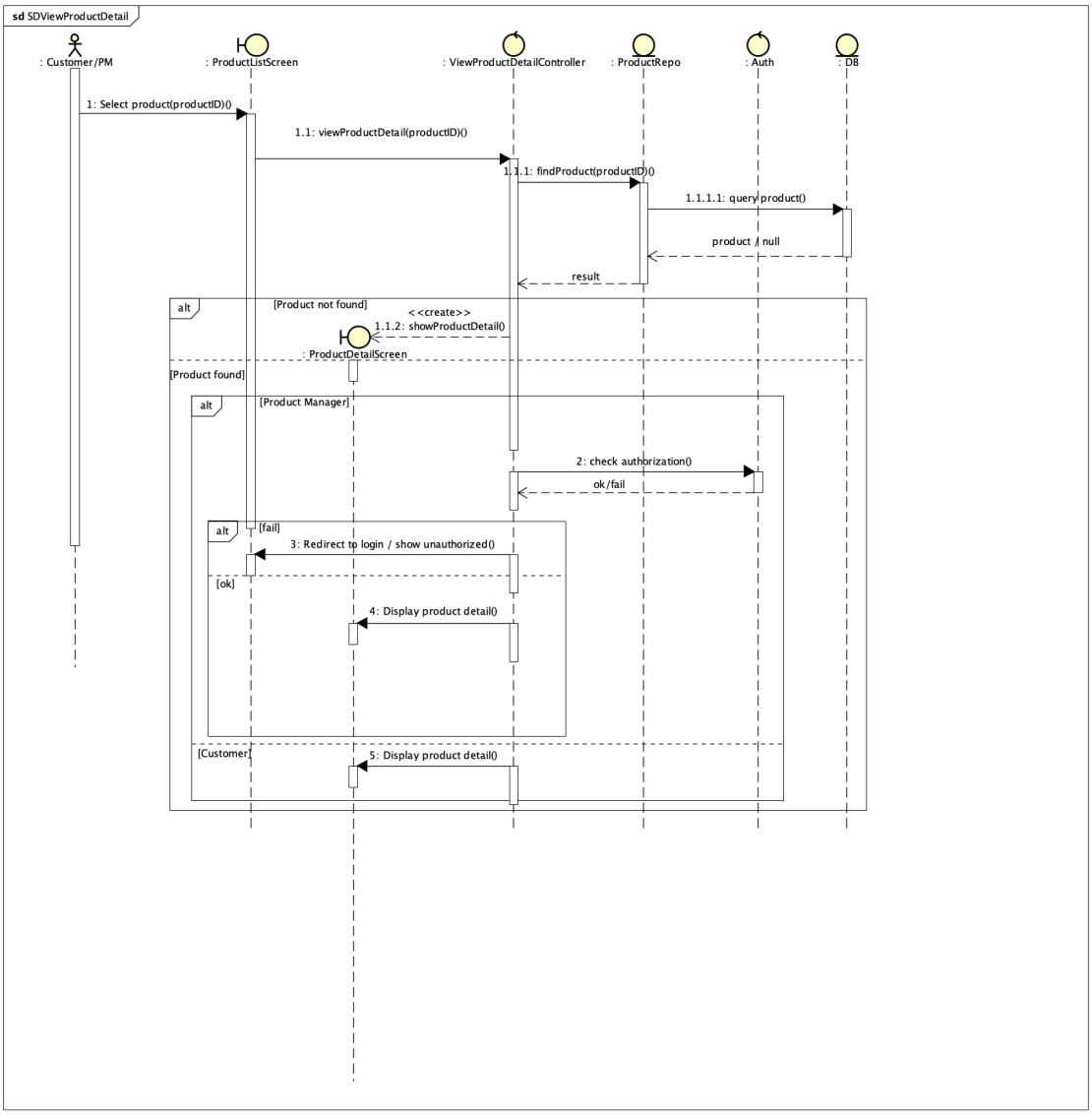

Figure 2. Sequence diagram – View Product Detail (UC235)

The communication (collaboration) diagram below presents the same interaction emphasising the structural links between the participating objects.

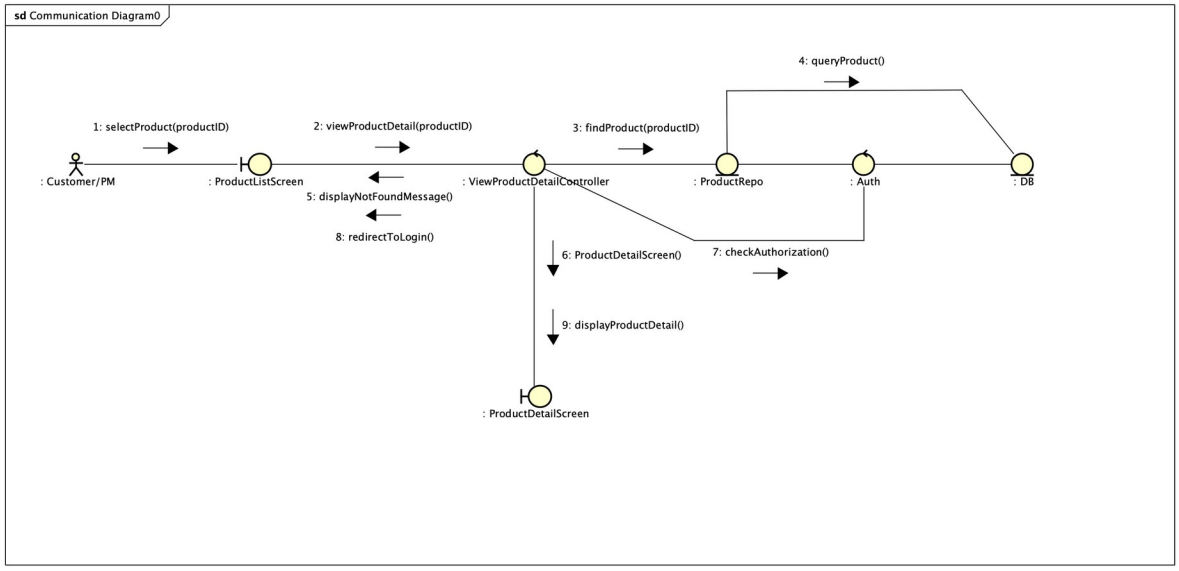

Figure 3. Communication diagram – View Product Detail

The refined interaction, expressed in Mermaid for reproducibility and aligned with the implemented layered design, is given below. It maps the analysis participants onto the concrete design classes (ProductController, GetProductDetailUseCase, ProductQueryRepository, ProductDetailMapper).

```
sequenceDiagram
  actor User as Customer / Product Manager
  participant UI as ProductDetailPageComponent
  participant Api as ProductApiService
  participant Ctrl as ProductController
  participant UC as GetProductDetailUseCase
  participant Repo as ProductQueryRepository
  participant Map as ProductDetailMapper
  participant DB as PostgreSQL (Prisma)
  User->>UI: select product (productId)
  UI->>Api: getProductDetail(productId)
  Api->>Ctrl: GET /products/:id
  Ctrl->>UC: execute(productId)
  UC->>UC: isValidProductId(productId)
  alt invalid id
    UC-->>Ctrl: BadRequestException
  else valid id
    UC->>Repo: findProductById(productId)
    Repo->>DB: SELECT ... JOIN subtype tables
    DB-->>Repo: product | null
    alt product is null
      UC-->>Ctrl: NotFoundException
      Ctrl-->>Api: 404
      Api-->>UI: error -> notFound = true (safe message)
    else product found
      UC->>Map: toDetailDto(product)
      Map-->>UC: ProductDetailDto
      UC-->>Ctrl: ProductDetailDto
      Ctrl-->>Api: 200 + JSON
      Api-->>UI: ProductDetail -> render detail screen
    end
  end
```

3.3 Analysis Class Diagrams

The analysis model for UC235 follows the boundary–control–entity decomposition. The boundary classes ProductListScreen and ProductDetailScreen represent the screens the actor interacts with. The control class ViewProductDetailController coordinates the flow, validating the request, resolving the actor role, and deciding whether to display the detail or a not-found/authorization message. The entity classes ProductRepo and DB encapsulate retrieval of product data, and the Auth control class checks authorization for the Product Manager.

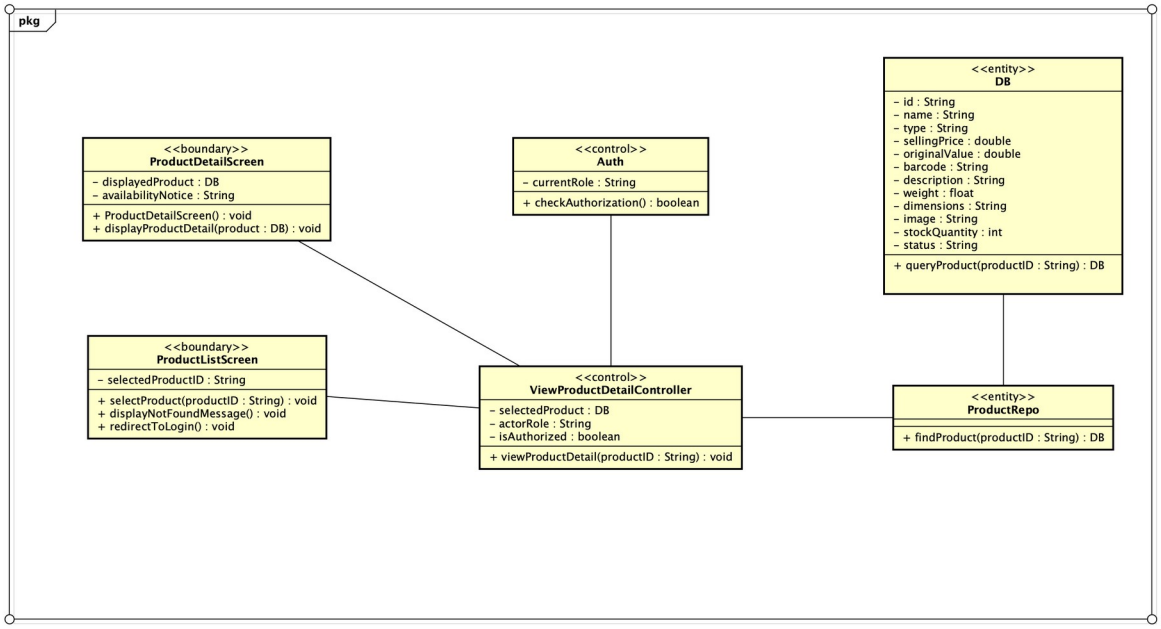

Figure 4. Analysis class diagram – View Product Detail

3.4 Unified Analysis Class Diagram

The unified analysis class diagram of AIMS integrates the per-use-case analysis classes into a single model in which shared entities (such as Product and its subtypes) are reused across use cases. The View Product Detail use case contributes the ViewProductDetailController control class and reuses the Product entity hierarchy that is also used by Place Order and CUD Product. The complete unified diagram is maintained collectively by the team in ArchitecturalDesign/AnalysisClassDiagram/Combined; the View Product Detail contribution to it is the boundary–control–entity slice shown in Figure 4.

3.5 Security Software Architecture

Security for AIMS is enforced at the API boundary through JSON Web Token (JWT) authentication and Role-Based Access Control (RBAC). Two NestJS guards implement the architecture: JwtAuthGuard authenticates the bearer token and populates the current user, and RolesGuard authorizes the request against the roles declared by the @Roles decorator. Read access to product detail (GET /products/:id) is intentionally public so that Customers can browse without logging in, whereas command endpoints (create, update, delete, upload image, view logs) are protected by JwtAuthGuard and RolesGuard with the role Product Manager.

For the View Product Detail use case, the access policy is therefore: a Customer may view any active product without authentication; a Product Manager is authenticated and may additionally view products regardless of sale availability. At the analysis level this corresponds to the Auth.checkAuthorization() step in Figure 2. The application layer adds defence-in-depth by validating the product identifier (rejecting non-positive or non-numeric values with a 400 response) and by never exposing raw Prisma records — only the read-only ProductDetailDto is returned, preventing over-exposure of internal fields.

TODO (proposed hardening): the View Product Detail availability rule for Customers (hiding deactivated products) is currently enforced through status normalisation in the mapper and the client. A dedicated authorization policy in the use case — rejecting Customer access to non-active products with a 403/404 — would make the rule explicit and centrally testable. This is captured by the ProductDetailAccessChecker class already present in the codebase but not yet wired into the main query path.

4 Detailed Design

4.1 User Interface Design

The View Product Detail feature is presented through a graphical user interface implemented as Angular standalone components. The Product Detail page is a smart page component (ProductDetailPageComponent) that reads the product identifier from the route, requests the data through ProductApiService, and delegates rendering to the presentational ProductDetailComponent and its type-specific child components (book, newspaper, CD, and DVD information panels).

4.1.1 Screen Configuration Standardization

All AIMS screens follow a common configuration: a fixed application header (AimsHeaderComponent) containing the brand, a global search field, primary navigation, and a cart indicator; a central content region; and a shared footer (AimsFooterComponent). Status and error states are rendered through a reusable StatusMessageComponent so that loading, not-found, and error messages are visually consistent across the application. The Product Detail screen reuses these standardized elements and adds a product-specific content region.

4.1.2 Screen Transition Diagrams

The diagram below shows how the View Detail Product screen is reached. The actor can navigate to it from the Homepage / Default Collection or from the Homepage / Filtered and Sorted Results (product list and search result) by selecting a product; the selected product data is passed to the detail screen. From the detail screen the actor can add the product to the cart (transitioning to the Shopping Cart) or navigate back to continue browsing.

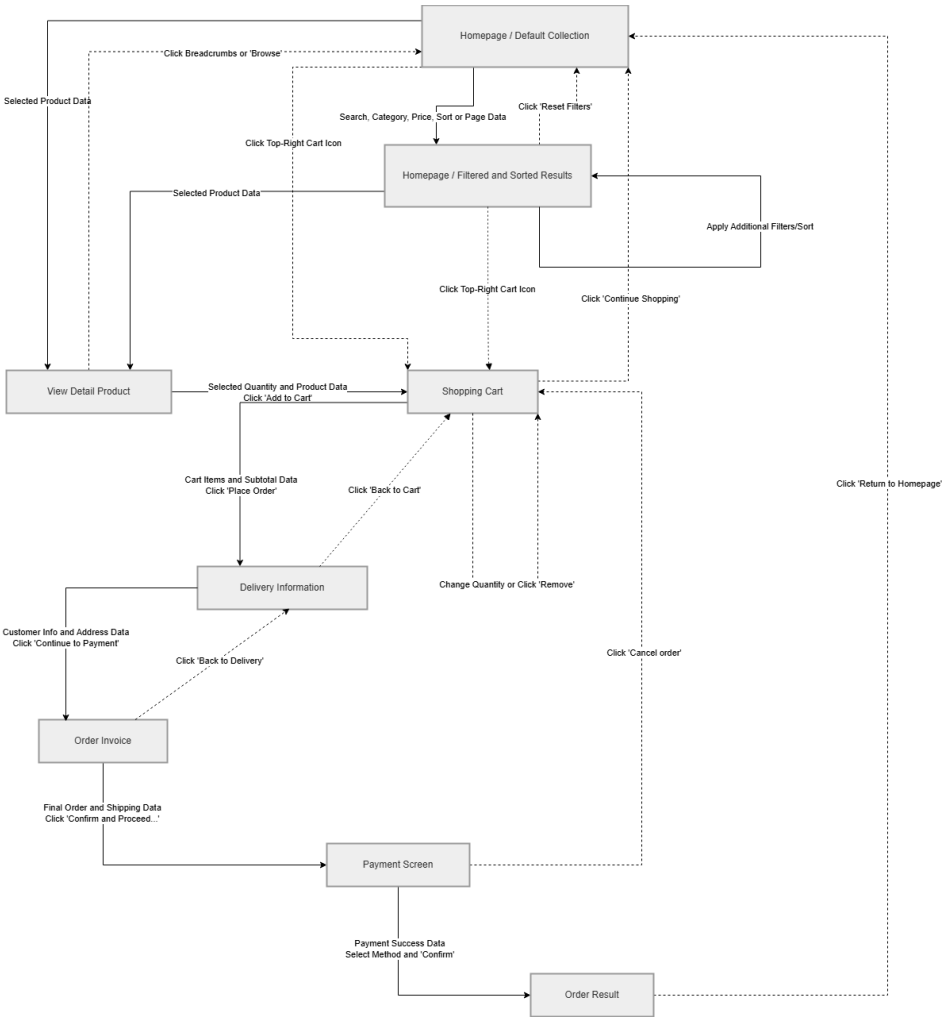

Figure 5. Screen transition diagram (View Detail Product highlighted)

4.1.3 Screen Specifications

The Product Detail screen displays the product image, breadcrumb navigation, the general product information (title, subtitle, type, selling price, availability badge), a quantity selector with an Add-to-Cart action, a panel of type-specific attributes, and an editorial description. The mockup below illustrates the screen for a CD product.

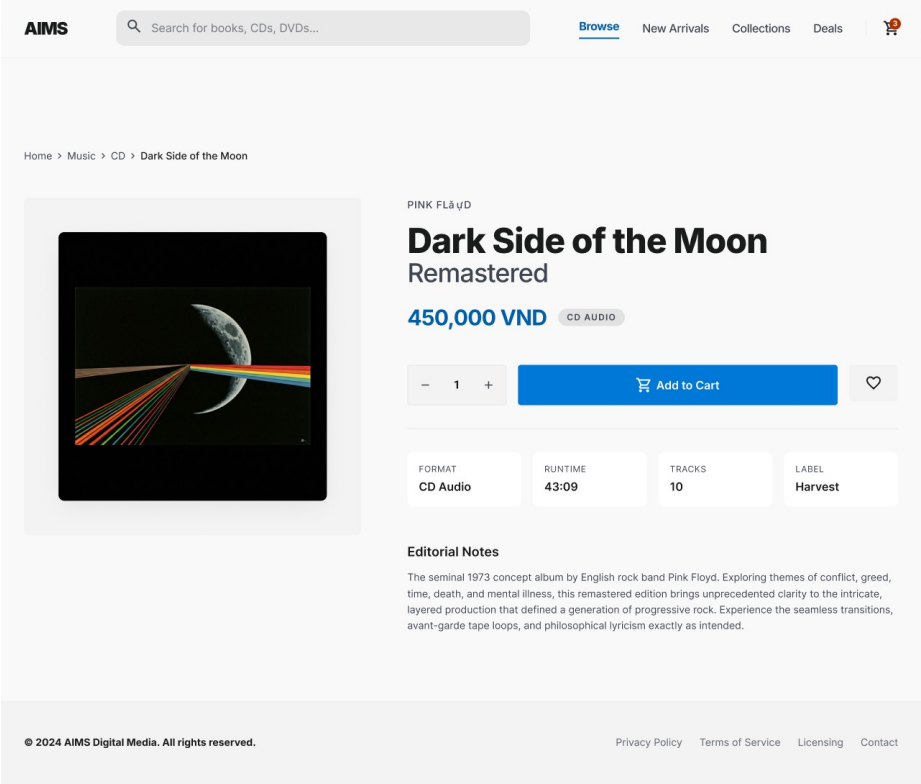

Figure 6. Product Detail screen mockup

Table 1. Screen specification of the Product Detail screen

No	Screen element	Description	Source field
1	Breadcrumb	Navigation path (e.g., Home / Music / CD / product title) allowing safe return to list.	type, title
2	Product image	Main visual of the product; placeholder shown when imageUrl is absent.	imageUrl
3	Title / subtitle	Main product name displayed as page heading.	title
4	Type badge	Category to which the product belongs (Book, Newspaper, CD, DVD).	type
5	Selling price	Current selling price formatted in VND.	currentPrice
6	Availability badge	Derived status: Available, Out of stock, or Deactivated.	status, stockQuantity
7	Quantity selector	Numeric stepper bounded by available stock.	stockQuantity
8	Add to Cart	Primary action; enabled only when the product can be purchased.	status, stockQuantity
9	Specific-info panel	Type-specific attributes (e.g., format, runtime, tracks, label for a CD).	specificInfo
10	Description	Editorial description / summary of the product.	description

4.2 Data Modeling

4.2.1 Conceptual Data Modeling

The conceptual data model represents products through a generalization hierarchy. The Product entity holds the attributes common to every item. A product is specialized either as a PrintableProduct (further specialized as Book or Newspaper) or as a DiscProduct (further specialized as CD or DVD). A Book is related to one or more Authors through the associative entity BookAuthor. This generalization captures the “general attributes + type-specific attributes” structure required by UC235.

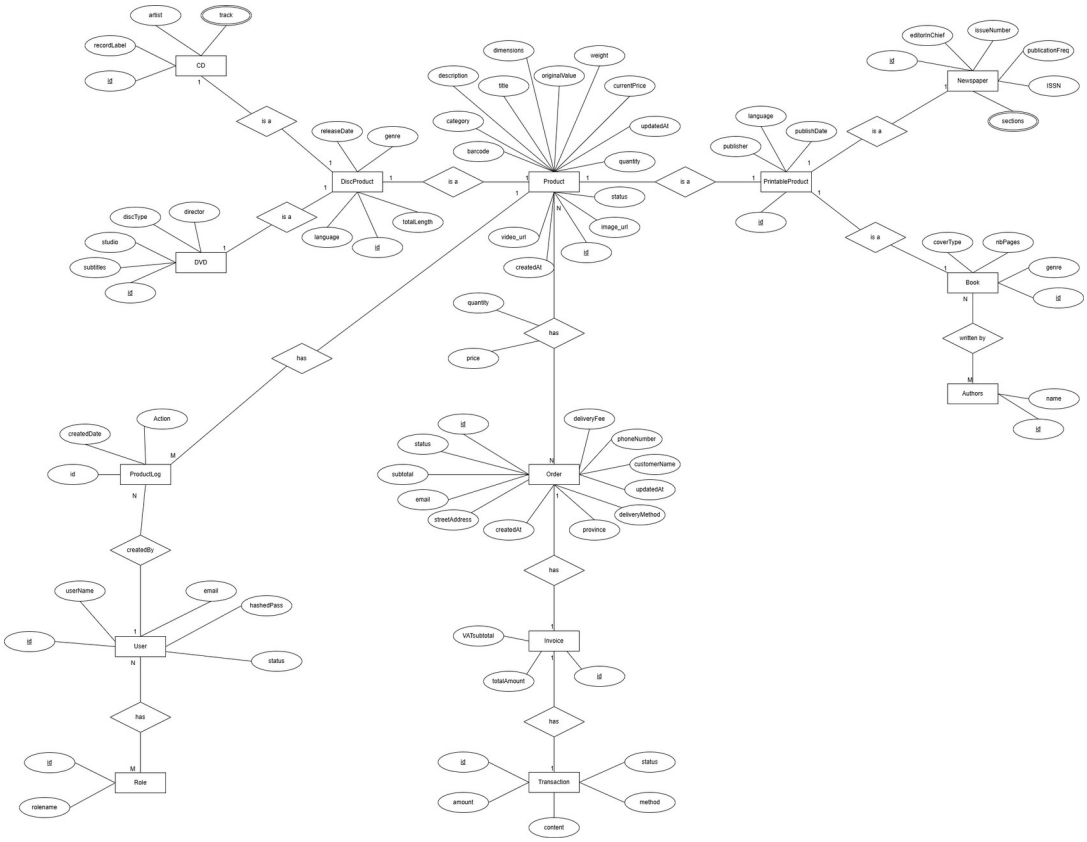

Figure 7. Entity–Relationship diagram (AIMS)

4.2.2 Database Design

4.2.2.1 Database Management System

AIMS uses PostgreSQL as its relational database management system, accessed through the Prisma ORM. PostgreSQL was selected for its strong support of relational integrity, transactions, and precise numeric types (the NUMERIC/Decimal type used for monetary values), which suit an e-commerce catalog. Prisma provides a type-safe query layer and generates the client consumed by the infrastructure repositories.

4.2.2.2 Database Diagram

The generalization hierarchy of the conceptual model is mapped to a class-table-inheritance schema: the root Product table holds shared columns, and each subtype is stored in its own

table sharing the same primary key as Product (one-to-one). The resulting physical schema is shown below.

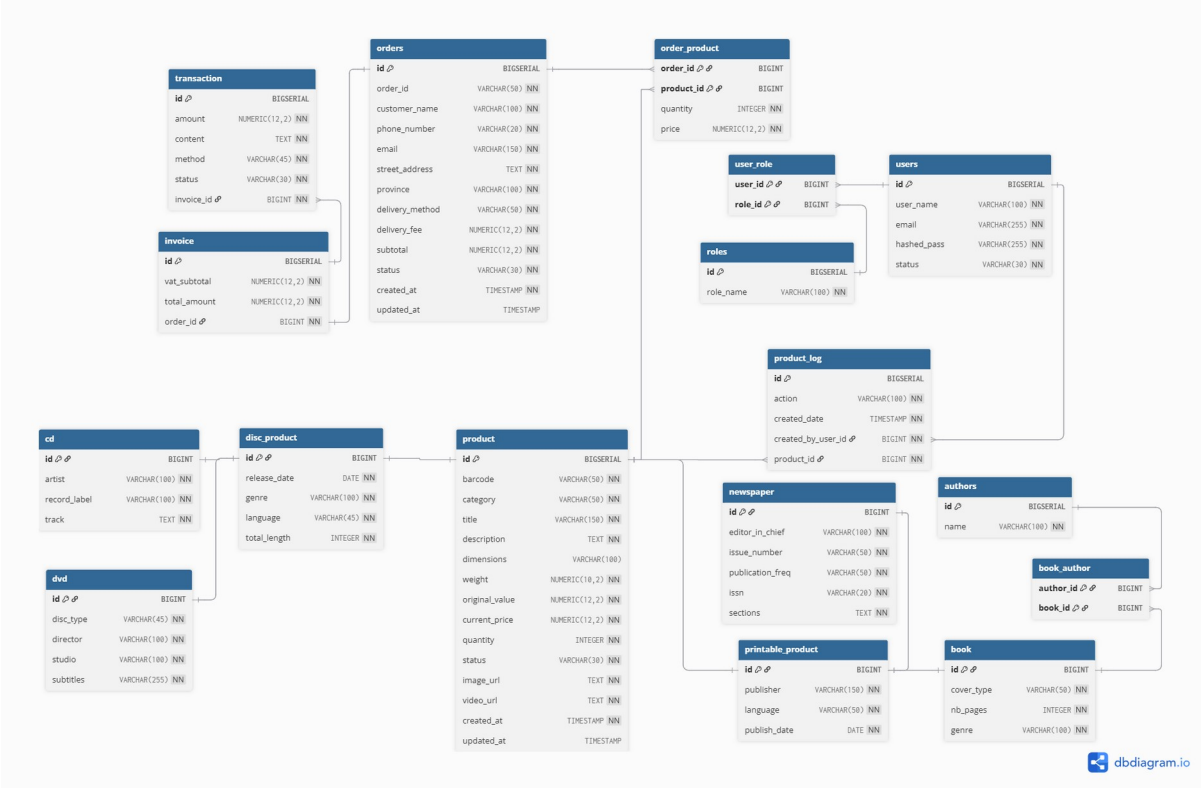

Figure 8. Database design diagram

4.2.2.3 Database Detail Design

The tables read when displaying a product detail are described below. A single product detail query joins product with its applicable subtype tables (printable_product → book/newspaper, or disc_product → cd/dvd, and book_author → authors for books).

Table 2. Table design – product (root table)

Column	Type	Constraint	Description
id	BIGINT	PK, identity	Unique product identifier (received as productId in UC235).
barcode	VARCHAR(50)	UNIQUE, NOT NULL	Unique product barcode.
category	VARCHAR(50)	NOT NULL	Product type discriminator (BOOK, NEWSPAPER, CD, DVD).
title	VARCHAR(150)	NOT NULL	Product name shown as the page title.
description	TEXT	NOT NULL	Editorial description / summary.
dimensions	VARCHAR(100)	NULL	Physical dimensions (height × width × length).
weight	DECIMAL(10,2)	NOT NULL	Physical weight of the product.
original_value	DECIMAL(12,2)	NOT NULL	Original value for management

Column	Type	Constraint	Description
			purposes.
current_price	DECIMAL(12,2)	NOT NULL	Current selling price shown to users.
quantity	INT	DEFAULT 0	Stock quantity available in inventory.
status	VARCHAR(30)	NOT NULL	Lifecycle status (e.g., ACTIVE) used to derive availability.
image_url	TEXT	NOT NULL	URL of the product image (external media service).
video_url	TEXT	NOT NULL	Optional promotional video URL.
created_at / updated_at	TIMESTAMP	created_at DEFAULT now()	Audit timestamps.

Table 3. Subtype and associative tables used by View Product Detail

Table	Key columns	Purpose in View Product Detail
printable_product	id (PK, FK→product.id), publisher, language, publish_date	Shared attributes of Book and Newspaper products.
book	id (PK, FK→printable_product.id), cover_type, nb_pages, genre	Book-specific attributes.
newspaper	id (PK, FK→printable_product.id), editor_in_chief, issue_number, publication_freq, issn, sections	Newspaper-specific attributes.
disc_product	id (PK, FK→product.id), release_date, genre, language, total_length	Shared attributes of CD and DVD products.
cd	id (PK, FK→disc_product.id), artist, record_label, track	CD-specific attributes.
dvd	id (PK, FK→disc_product.id), disc_type, director, studio, subtitles	DVD-specific attributes.
authors / book_author	authors(id, name); book_author(book_id, author_id)	Many-to-many link giving the authors of a book.

Indexing & integrity: product.id is the primary key and the natural lookup key for UC235; product.barcode carries a unique index. Each subtype table shares the primary key of its parent and declares a foreign key with ON DELETE CASCADE, so deleting a product removes its subtype rows and preserves referential integrity. No trigger or view is required for the read-only detail query.

4.3 Non-Database Management System Files

Product media (images and optional videos) are not stored in the database. They are uploaded to and served from an external media service (Cloudinary); the database stores only the

resulting URL in product.image_url and product.video_url. During View Product Detail the client requests these binary resources directly from the media service CDN by URL. These files are read-only for this use case, are updated only through the Product Manager's upload-image endpoint, and are backed up by the external service; AIMS therefore keeps no local non-DBMS record structures for them.

4.4 Class Design

4.4.1 General Class Diagram

The general design class diagram of the product feature shows the participating classes grouped by layer: the boundary/UI classes on the client, the ProductController in the interface layer, the GetProductDetailUseCase, ProductDetailMapper, and ProductDetailDto in the application layer, the ProductQueryRepository interface and entities in the domain layer, and the PrismaProductRepository in the infrastructure layer.

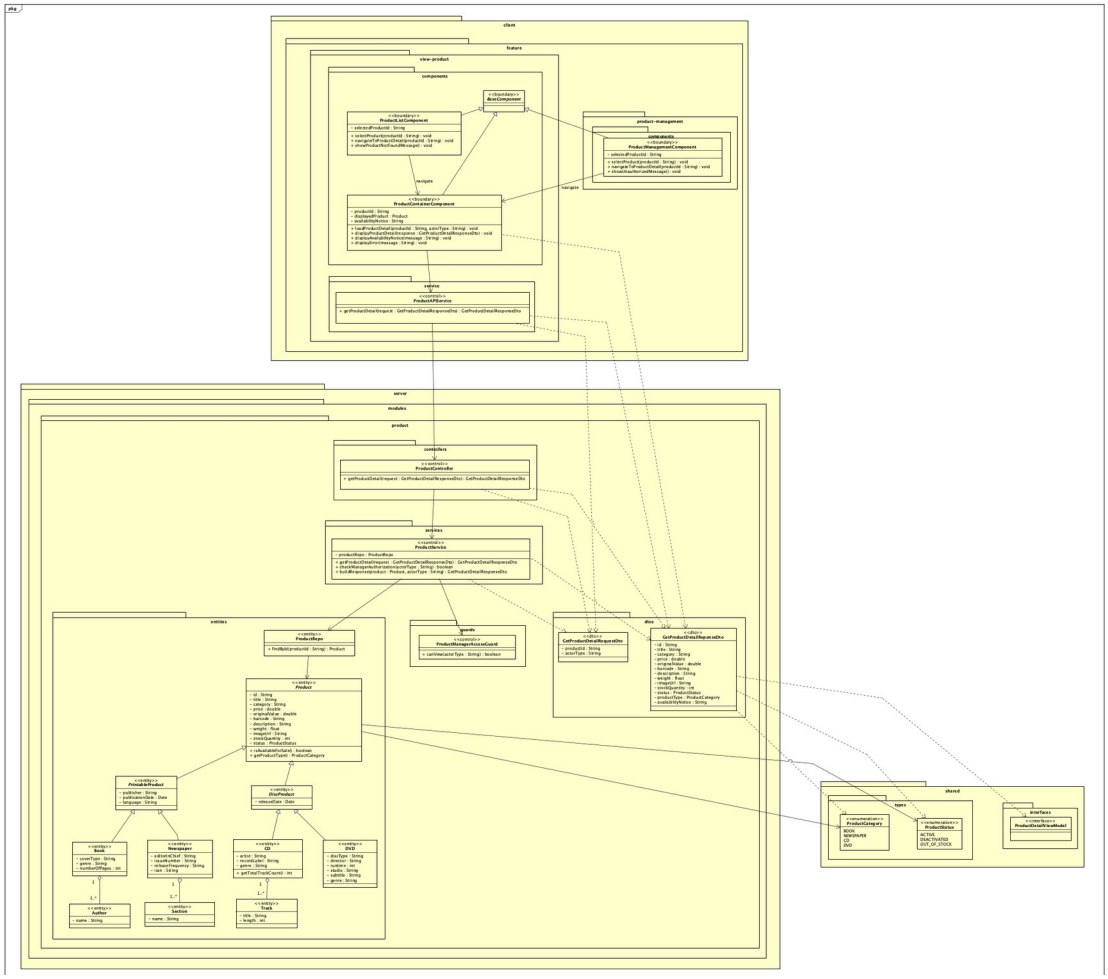

Figure 9. Design class diagram – View Product Detail

4.4.2 Class Diagrams

4.4.2.1 Class Diagram for the Product (View Detail) Package

The detailed class diagram in Figure 9 captures the attributes and operations of each class involved in UC235. The application layer depends on the ProductQueryRepository abstraction (Dependency Inversion); PrismaProductRepository implements it; and ProductDetailMapper transforms the persistence model into the ProductDetailDto consumed by the client. Each class is detailed in Section 4.4.3.

4.4.3 Class Design

4.4.3.1 Class "GetProductDetailUseCase"

Application-layer control class that orchestrates UC235. It validates the product identifier, retrieves the product through the repository abstraction, raises the appropriate exception when the identifier is invalid or the product does not exist, and returns a ProductDetailDto produced by the mapper.

Table 4. Attribute design – GetProductDetailUseCase

Attribute	Type	Description
productRepository	ProductQueryRepository	Injected repository abstraction (DIP) used to read product data.

Table 5. Operation design – GetProductDetailUseCase

Operation	Return	Description
execute(productId)	Promise<ProductDetailDto>	Validates the id, finds the product, maps it to a DTO; throws when invalid/not found.
isValidProductId(productId)	boolean	Private guard accepting only positive integer identifiers (/^[1-9]\d*$/).

Parameter:

- productId: string — the identifier received from the controller route parameter.

Exception:

- BadRequestException — if the identifier is not a positive integer.
- NotFoundException — if no product with the given identifier exists.

Method: the use case first calls isValidProductId; on success it awaits productRepository.findProductById and, if a record is returned, delegates to ProductDetailMapper.toDetailDto. No state is held between calls.

4.4.3.2 Class "ProductQueryRepository" (interface)

Domain-layer abstraction exposing only the read operations required by product viewing. It defines the contract the application layer depends upon and that the infrastructure layer implements, satisfying the Interface Segregation and Dependency Inversion principles.

Table 6. Operation design – ProductQueryRepository

Operation	Return	Description
findProducts()	Promise<Model[]>	Returns the product list projection used by the catalog screen.
findProductById(productId)	Promise<Model | null>	Returns a single product with its subtype relations, or null if absent.

4.4.3.3 Class “PrismaProductRepository”

Infrastructure-layer class implementing ProductQueryRepository with Prisma. findProductById converts the string identifier to BigInt and issues a single findUnique query that includes the printable (book → authors, newspaper) and disc (cd, dvd) relations, so that all data needed to display any product type is retrieved in one round trip. Prisma models never leave this layer; mapping to DTOs is performed by ProductDetailMapper.

```
findProductById(productId): Promise<Model|null> {
  return this.prisma.product.findUnique({
    where: { id: BigInt(productId) },
    include: {
      printableProduct: { include: { book: { include: { bookAuthors: { include: { author: true } } } },
newspaper: true } },
      discProduct: { include: { cd: true, dvd: true } },
    },
  });
}
```

4.4.3.4 Class “ProductDetailMapper”

Application-layer mapper that transforms a persistence record into the read-only ProductDetailDto. It normalizes the type and status, parses dimensions, and dispatches to a type-specific mapping function for the specificInfo block. This class concentrates all transformation logic so that controllers and the client never depend on Prisma model shapes.

Table 7. Operation design – ProductDetailMapper

Operation	Return	Description
toDetailDto(model)	ProductDetailDto	Maps general fields and delegates type-specific fields to mapSpecificInfo.
mapSpecificInfo(type, model)	ProductSpecificInfoDto	Selects the mapping function for the product type (Book/Newspaper/CD/DVD).
normalizeStatus(status, qty)	string	Derives OUT_OF_STOCK from an active product with zero stock.
parseDimensions(value)	ProductDimensions	Robustly parses object/JSON/“H×W×L” dimension formats.

4.4.3.5 Class “ProductDetailDto”

The response contract returned to the client. It carries the general product fields and a discriminated specificInfo object so that new product types extend the specific section without altering the common contract.

Table 8. Attribute design – ProductDetailDto

Attribute	Type	Description
id, barcode, title	string	Identity and name fields.
type	ProductTypeValue	BOOK | NEWSPAPER | CD | DVD.
currentPrice, originalValue, weight	number	Monetary and physical values.
description	string	Editorial description.
dimensions	ProductDimensions	Height, width, length.
imageUrl	string?	Optional image URL.
stockQuantity	number	Available stock.
status	string	Derived availability status.
specificInfo	ProductSpecificInfoDto	Type-specific attributes.

4.4.3.6 Class "ProductController"

Interface-layer class that maps the HTTP endpoint GET /products/:id to the use case. The handler findOne(id) is intentionally thin: it delegates entirely to getProductDetailUseCase.execute(id) and performs no querying, mapping, or formatting. Read access is public; command endpoints in the same controller are protected by JwtAuthGuard and RolesGuard.

Table 9. Operation design – ProductController (View Product Detail handler)

Operation	HTTP	Description
findOne(id)	GET /products/:id	Returns the ProductDetailDto for the given product, or a 404 when absent.

4.4.3.7 Class "ProductApiService" (client)

Client-side service that centralizes product read endpoints. getProductDetail(productId) issues the HTTP GET and normalizes the response into the ProductDetail model, tolerating field-name variations from the back-end. Components depend on this service rather than on HttpClient directly.

4.4.3.8 Class "ProductDetailPageComponent" (client)

Smart page component that reads the route parameter, manages loading/error/not-found signals, calls ProductApiService.getProductDetail, and delegates rendering to ProductDetailComponent. On a 404 response it sets notFound and offers a safe navigation back to the catalog, implementing the alternative flow of UC235 on the client.

5 Design Considerations

5.1 Goals and Guidelines

The dominant design goals for View Product Detail are maintainability and testability without sacrificing the SRS response-time targets. The design emphasizes a clear separation of concerns so that a change in persistence, presentation, or business rules affects only one layer. Coding guidelines follow the team conventions enforced by ESLint/Prettier and commit-lint: classes expose narrow public interfaces, transformation logic is isolated in mappers, and each class carries an explicit SOLID review comment in the source. Readability and a consistent screen configuration are prioritized so the feature “feels” the same as the rest of AIMS.

5.2 Architectural Strategies

- Technology choices: NestJS dependency injection enables the Dependency Inversion strategy; Prisma provides type-safe persistence; Angular signals manage UI state with minimal boilerplate.
- Reuse: the same ProductQueryRepository and ProductDetailMapper serve both the list and the detail screens, avoiding duplicated query and mapping logic.
- Extensibility: type-specific behavior is isolated so new product types can be added with localized changes (further improved by the Strategy pattern in Section 5.5).
- Error detection and recovery: identifier validation, explicit not-found handling, and a safe client redirect ensure graceful degradation.
- Persistence: a single included query retrieves all subtype data in one round trip to meet the response-time budget.

5.3 Coupling and Cohesion

Each class participating in View Product Detail was evaluated for cohesion and coupling. The design consistently reaches functional cohesion (each class performs one well-defined task) and data/stamp coupling (classes exchange simple data or DTOs rather than sharing control or global state). The table below summarizes the evaluation.

Table 10. Cohesion & SRP of the View Product Detail classes

Class	Cohesion	Coupling	Justification
ProductController.findOne	Functional	Data	Performs the single task of mapping the HTTP request to the use case; passes only the id.
GetProductDetailUseCase	Functional	Data	Coordinates one business flow; depends on the repository abstraction and exchanges a string id and a DTO.
ProductQueryRepository	Functional	Data	Defines only read operations; no implementation coupling.
PrismaProductRepository	Functional	Stamp	Single responsibility (Prisma read); returns a structured record consumed only by the mapper.

Class	Cohesion	Coupling	Justification
ProductDetailMapper	Functional	Stamp	Only transforms records to DTOs; receives the whole record but uses selected fields.
ProductDetailDto	Functional	Data	Pure data contract with no behavior or dependencies.
ProductDetailPageComponent	Functional	Data	Coordinates route/state/render; delegates data access and presentation.

5.4 Design Principles

The design is evaluated against the SOLID principles to assess its resilience to future changes (for example, adding a new product type such as LP record, or a second persistence implementation).

Table 11. Coupling & other SOLID principles of AIMS (View Product Detail)

Principle	Status	Evidence in the View Product Detail design
SRP	Satisfied	Controller (HTTP), use case (orchestration), repository (persistence), mapper (transformation), and DTO (contract) each have one reason to change.
OCP	Satisfied (improved)	New product types are added by extending specificInfo mapping; the proposed Strategy registry (5.5) removes the remaining type switch so existing code is not modified.
LSP	Satisfied	Any ProductQueryRepository implementation can replace Prisma; all product subtypes preserve the common ProductDetailDto contract.
ISP	Satisfied	Read clients depend on ProductQueryRepository (query-only); command operations live in separate abstractions.
DIP	Satisfied	Application/interface layers depend on the repository abstraction via the PRODUCT_QUERY_REPOSITORY token, not on Prisma.

Improvement applied: the original detail path exposed Prisma records and concentrated all type handling in conditional code. The refactored design introduces the ProductQueryRepository abstraction, a dedicated ProductDetailMapper, and a read-only ProductDetailDto. This moved the design from content/control coupling toward data/stamp coupling and from a single multi-responsibility service toward functionally cohesive classes, while preserving the existing GET /products/:id behavior.

5.5 Design Patterns

Several patterns are applied. The Repository pattern (ProductQueryRepository / PrismaProductRepository) decouples business logic from persistence. The Data Transfer Object and Mapper patterns (ProductDetailDto / ProductDetailMapper) prevent persistence models from leaking outward. The Dependency Injection pattern, provided by NestJS, wires abstractions to implementations through tokens.

To further improve the design, the Strategy pattern is proposed for type-specific mapping. The current ProductDetailMapper selects a mapping function by product type using an internal lookup. Extracting each branch into an IProductSpecificInfoMapper strategy, registered in a ProductSpecificInfoMapperRegistry, makes the mapper fully open for extension and closed for modification: adding a new product type means adding one strategy class and one registry entry, with no change to existing mappers. The proposed structure is shown below.

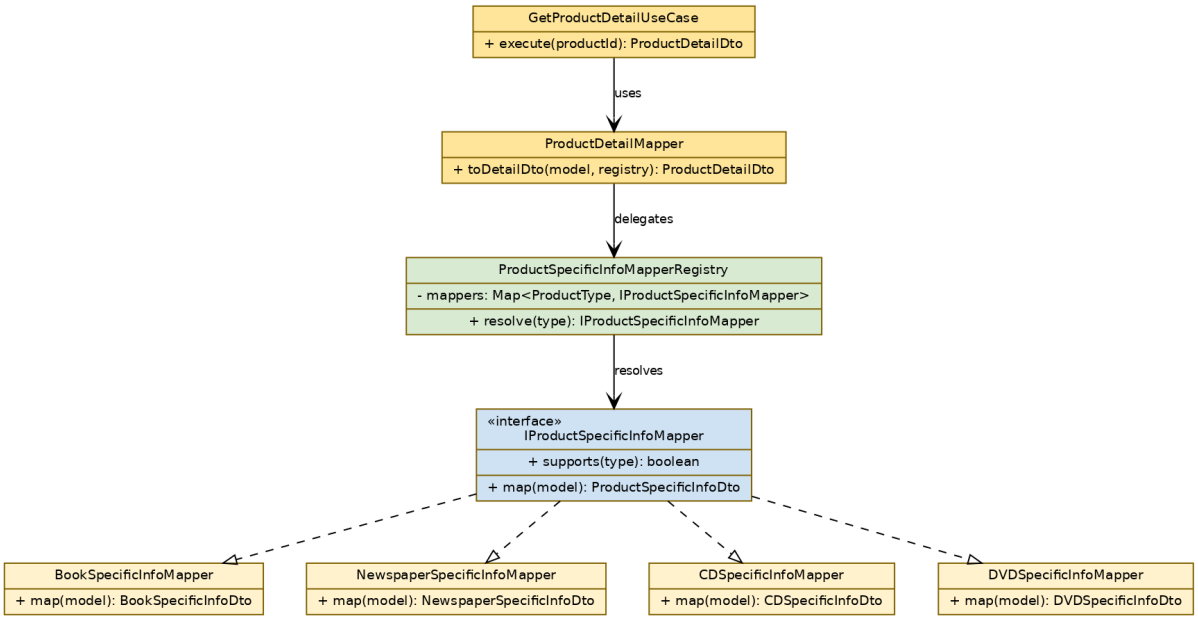

Figure 10. Proposed Strategy pattern for type-specific product mapping

TODO: introduce IProductSpecificInfoMapper, the four concrete strategies, and ProductSpecificInfoMapperRegistry, then have ProductDetailMapper.toDetailDto resolve the strategy from the registry. This is a non-breaking refactor — the public ProductDetailDto and the GET /products/:id contract remain unchanged.